Cloud Computing

CAT7002 Course Notes

One part per unit. Unit 1: virtualization concepts and hypervisors.

by Bhuvnesh Verma

Contents

Unit 1: Virtualization Concepts and Hypervisors

1. Introduction to Servers

Short description: A server is a computer or software system that provides services, resources, or data to other computers, called clients, over a network.

Everyday examples:

- Opening a website: the browser sends a request to a **web server**.
- Checking Gmail: a **mail server** delivers the messages.
- Using online banking: a **database server** stores and retrieves the account.

What a server does: Stores files, hosts websites, manages databases, runs applications, sends and receives email, shares printers and other resources, and manages users and security.

1.1 Types of servers

Server type	Purpose
Web server	Hosts websites
File server	Stores and shares files
Database server	Stores databases
Mail server	Sends and receives emails
Application server	Runs business applications
Print server	Manages printers
DNS server	Converts domain names into IP addresses

2. The Traditional Server Model

Short description: Before virtualization and cloud computing, organizations typically deployed **one physical server for one application**. Each application had its own dedicated hardware.

Application	Physical server
Website	Server 1
Database	Server 2
Email	Server 3
ERP	Server 4

2.1 Advantages

- **High performance.** Applications have direct access to hardware resources.
- **Better security.** Each application runs on its own physical machine.
- **Reliability.** No interference from other applications sharing the hardware.
- **Simple architecture.** Easy to install, configure and troubleshoot.

- **Full hardware control.** Administrators control the server hardware completely.

2.2 Disadvantages

- **Low resource utilization.** Most servers use only 10–20% of CPU and memory, leaving the rest idle.
- **High cost.** Each application requires separate hardware.
- **High power consumption.** More servers draw more electricity and need more cooling.
- **More physical space.** Multiple servers occupy valuable rack space.
- **Difficult scalability.** A new application often means buying new hardware.
- **Higher maintenance.** Every server needs updates, monitoring, backups and repairs.
- **Single point of failure.** If a physical server fails, its application becomes unavailable.

Why a new technology was needed

As organizations expanded they faced too many physical servers, rising hardware costs, increased electricity and cooling expenses, underutilized resources, complex management, and slow deployment of new services. These limitations led to **virtualization**, which allows multiple virtual servers to run on a single physical machine, and virtualization in turn became the foundation for **cloud computing**.

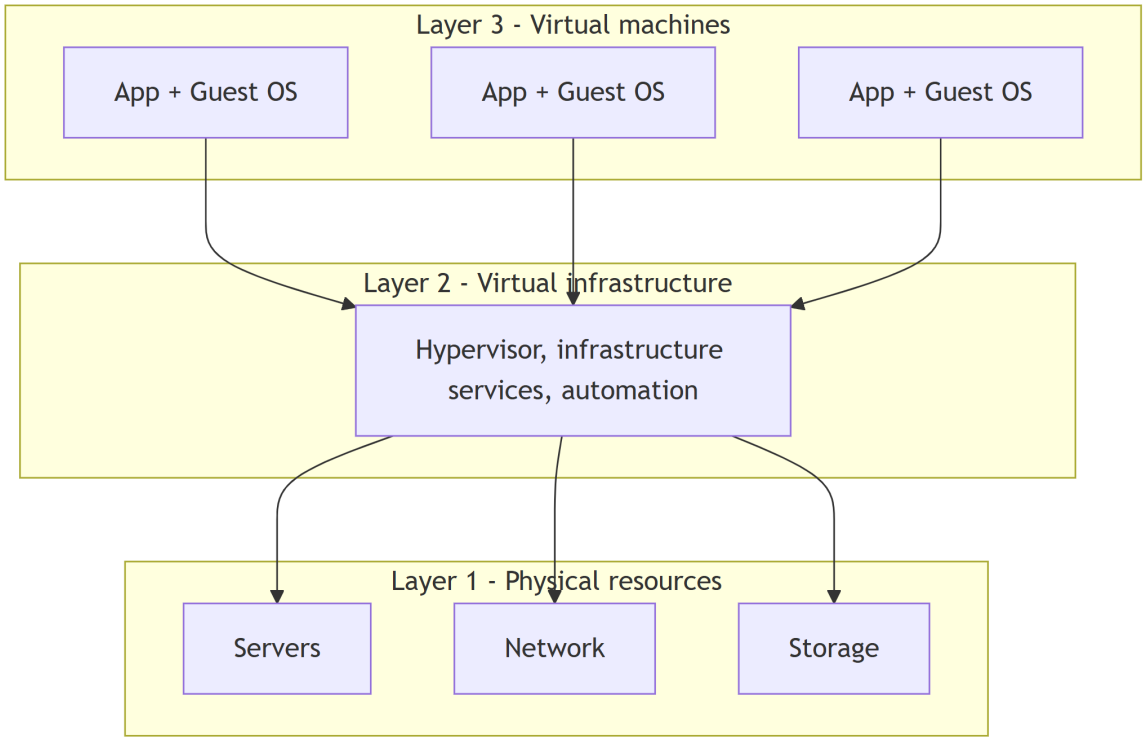
3. Introduction to Virtualization

Short description: Virtualization is a methodology for dividing computer resources into more than one execution environment, applying concepts such as partitioning, time-sharing, machine simulation and emulation.

Theory: Virtualization is a method in which multiple independent operating systems run on one physical computer. It maximizes the usage of available physical resources, so one can achieve high server usage: it increases total computing power while decreasing overhead. Increasing the number of logical operating systems on a single host reduces hardware acquisition and maintenance cost for an organization.

How it works: The architecture has three layers. Layer 1 is the physical infrastructure of servers, networks and storage. Layer 2 is the virtual infrastructure, consisting of hypervisors, virtual infrastructure services and automated solutions that optimize the IT process. Layer 3 holds the virtual machines, where different operating systems and applications are deployed. A single virtual infrastructure supports more than one virtual machine, so the physical resources of the whole infrastructure are shared across many virtual machines for maximum efficiency.

3.1 Architecture diagram



Notes

- Virtualization is a trend in IT that includes autonomic computing and utility computing: the environment manages itself, and every resource is seen as a utility that a client pays per use.
- It reduces the burden of workloads on users by centralizing administrative tasks and improving scalability.

3.2 Reasons for using virtualization

- Virtual machines consolidate the workloads of under-utilized servers, saving on hardware, environmental costs and management.
- Legacy applications can keep running inside a VM.
- A VM provides a secure sandbox for running an untrusted application.
- A VM helps in building a secured computing platform.
- A VM provides an illusion of hardware.
- A VM simulates networks of independent computers.
- A VM supports running distinct operating systems with different versions.

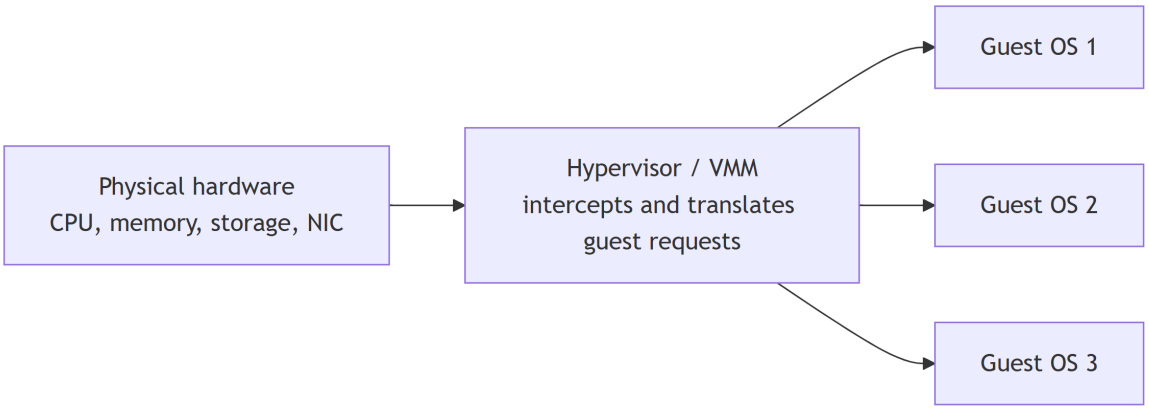
4. Hypervisor

Short description: A hypervisor, or Virtual Machine Monitor (VMM), is software that lets multiple operating systems run on a single physical machine.

Theory: It manages hardware resources such as CPU, memory and storage and allocates them to virtual machines without interference. This improves hardware utilization, reduces costs, and provides flexibility in cloud and server environments.

How it works: A hypervisor runs on hardware or on a host OS to create and manage virtual machines, each with its own virtual CPU, memory, storage and network. It intercepts guest OS requests and translates them to physical hardware, ensuring isolation, security and stability.

4.1 Architecture diagram



4.2 Type 1 hypervisor

A Type 1 hypervisor runs directly on the host's hardware and does not rely on a host operating system. This architecture offers better performance and security because there is no intermediary OS. It is the standard for enterprise-level data centres and for cloud providers such as Amazon Web Services and Microsoft Azure.

Examples: VMware ESXi, Microsoft Hyper-V, KVM (Kernel-based Virtual Machine), Xen.

- **Pros:** high performance through direct hardware access; strong security with no intermediate OS layer; suitable for mission-critical workloads.
- **Cons:** requires dedicated hardware; setup and management are complex compared to Type 2.

4.3 Type 2 hypervisor

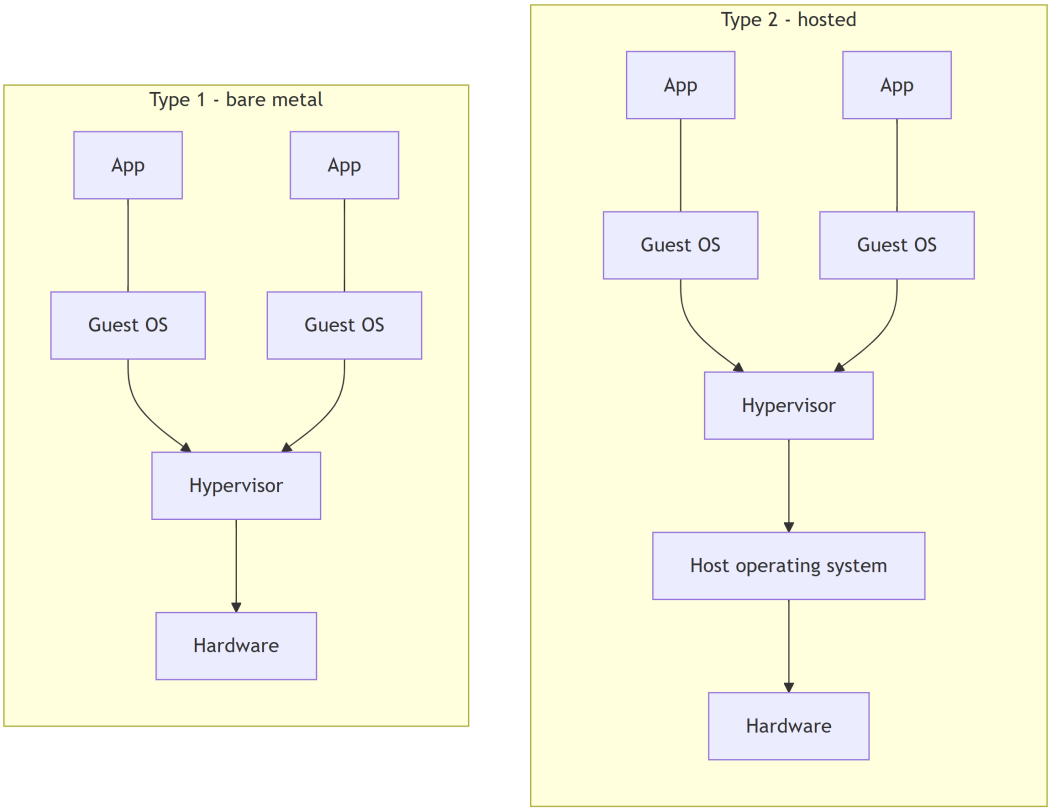
A Type 2 hypervisor runs on top of a conventional operating system such as Windows, macOS or Linux. It is essentially an application within the host OS. This type is generally used for desktop virtualization, development and testing, where a user needs to run multiple operating systems on a personal computer. Performance is slightly lower than Type 1 because of the overhead of the host OS.

Examples: Oracle VM VirtualBox, VMware Workstation, Parallels Desktop.

- **Pros:** easy to install and use; useful for development, testing and malware analysis; good host–guest integration features.

- **Cons:** slower performance with no direct hardware access; security depends on the host OS, so a compromise of the host may affect the guests.

4.4 Type 1 versus Type 2



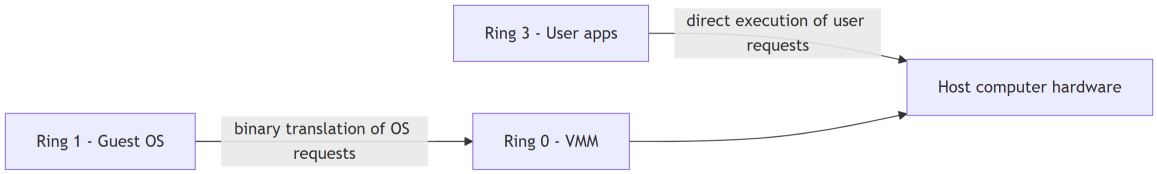
5. Full Virtualization

Short description: Full virtualization was introduced by IBM in 1966. It was the first software solution for server virtualization and uses binary translation together with direct execution.

How it works: The virtual machine completely isolates the guest OS from the virtualization layer and the hardware. Kernel code is translated to replace non-virtualizable instructions with new sequences of instructions that have the intended effect on the virtual hardware, while user level code is executed directly on the processor for high performance.

Examples: Microsoft and Parallels systems.

5.1 Architecture diagram



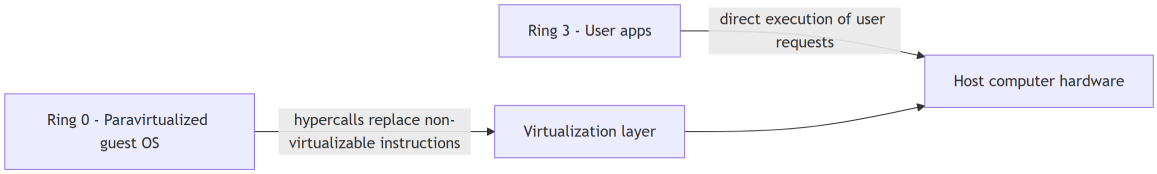
6. Paravirtualization

Short description: Paravirtualization is the category of CPU virtualization that uses hypercalls for operations handled at compile time.

How it works: The guest OS is not completely isolated, only partially isolated by the virtual machine from the virtualization layer and hardware. The OS kernel is modified to replace non-virtualizable instructions with hypercalls that communicate directly with the virtualization layer, which improves performance and efficiency.

Examples: VMware and Xen.

6.1 Architecture diagram

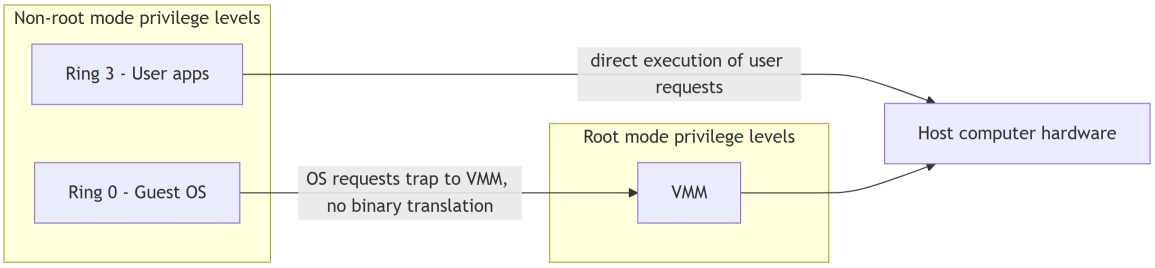


7. Hardware Based Virtualization

Short description: Hardware vendors added CPU features that simplify virtualization directly in silicon.

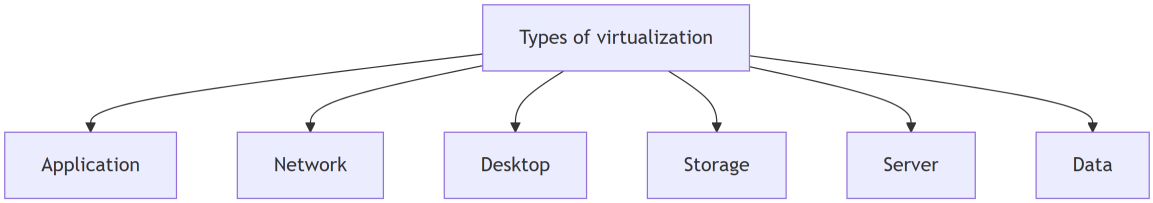
How it works: First generation enhancements include Intel Virtualization Technology (VT-x) and AMD's AMD-V. Both target privileged instructions with a new CPU execution mode that allows the VMM to run in a root mode below ring 0. Privileged and sensitive calls are set to trap automatically to the hypervisor, removing the need for either binary translation or paravirtualization. The guest state is stored in Virtual Machine Control Structures (VT-x) or Virtual Machine Control Blocks (AMD-V).

7.1 Architecture diagram



8. Types of Virtualization

8.1 Overview

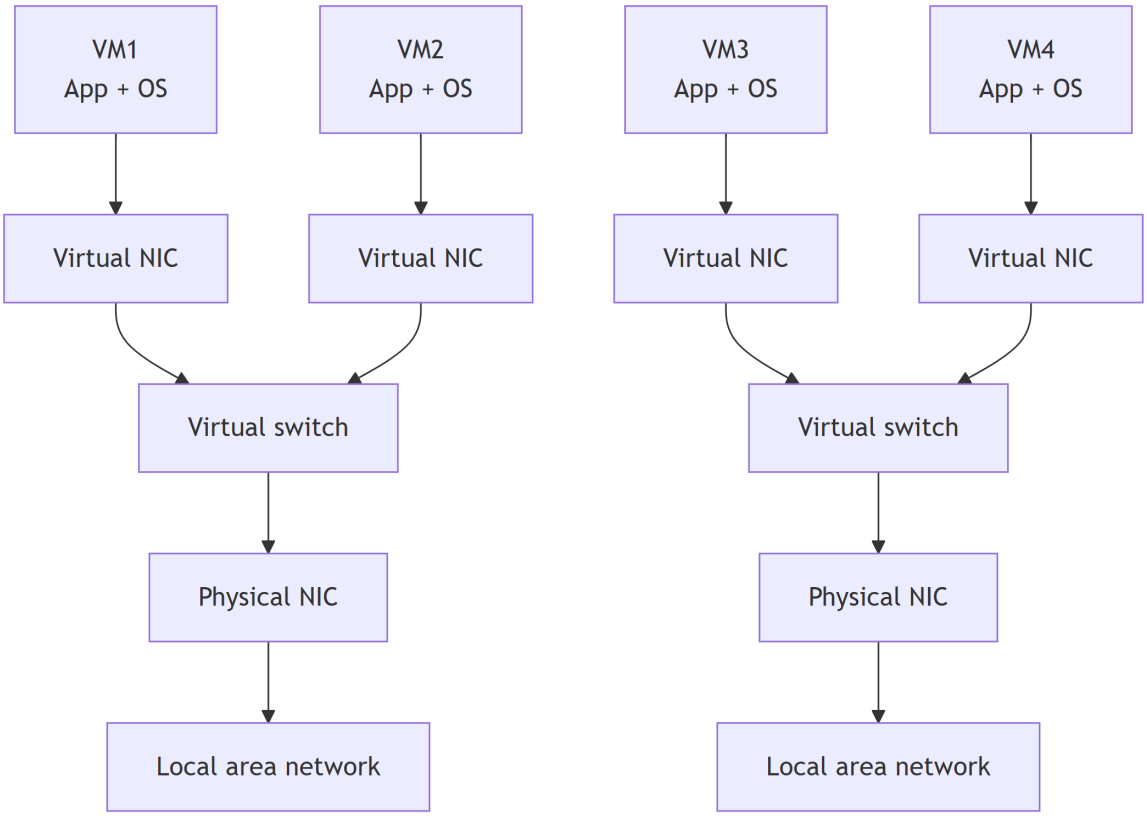


8.2 Desktop virtualization

Also called client virtualization. The virtualization happens on the user's site: desktops are replaced with thin clients that use datacentre resources.

8.3 Network virtualization

Part of the virtualization infrastructure, used especially when virtualizing servers. It allows multiple switches, VLANs and NAT-ing to be created in software.

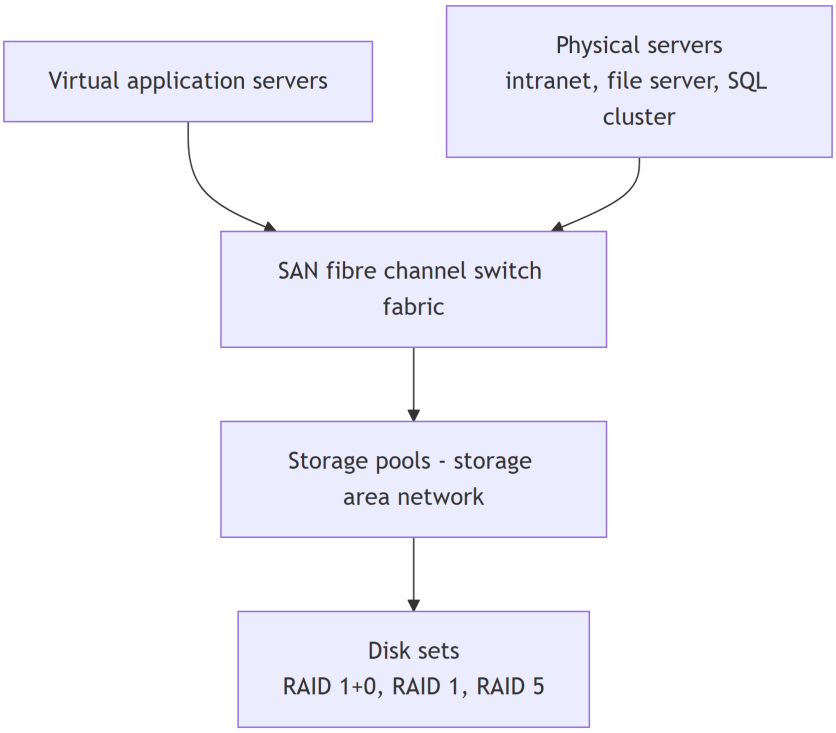


8.4 Server and machine virtualization

Virtualizing the server infrastructure, so no further physical servers are needed for different purposes.

8.5 Storage virtualization

Widely used in datacentres with large storage. It helps create, delete and allocate storage to different hardware, over a network connection. The leading technology here is the SAN.



8.6 Operating system level virtualization

An abstraction layer between the traditional OS and user applications. OS-level virtualization creates isolated **containers** on a single physical server, using one OS instance to utilize the hardware and software in data centres. The containers behave like real servers. It is commonly used to create virtual hosting environments that allocate hardware resources among a large number of mutually distrusting users, and to a lesser extent to consolidate server hardware by moving services on separate hosts into containers or VMs on one server.

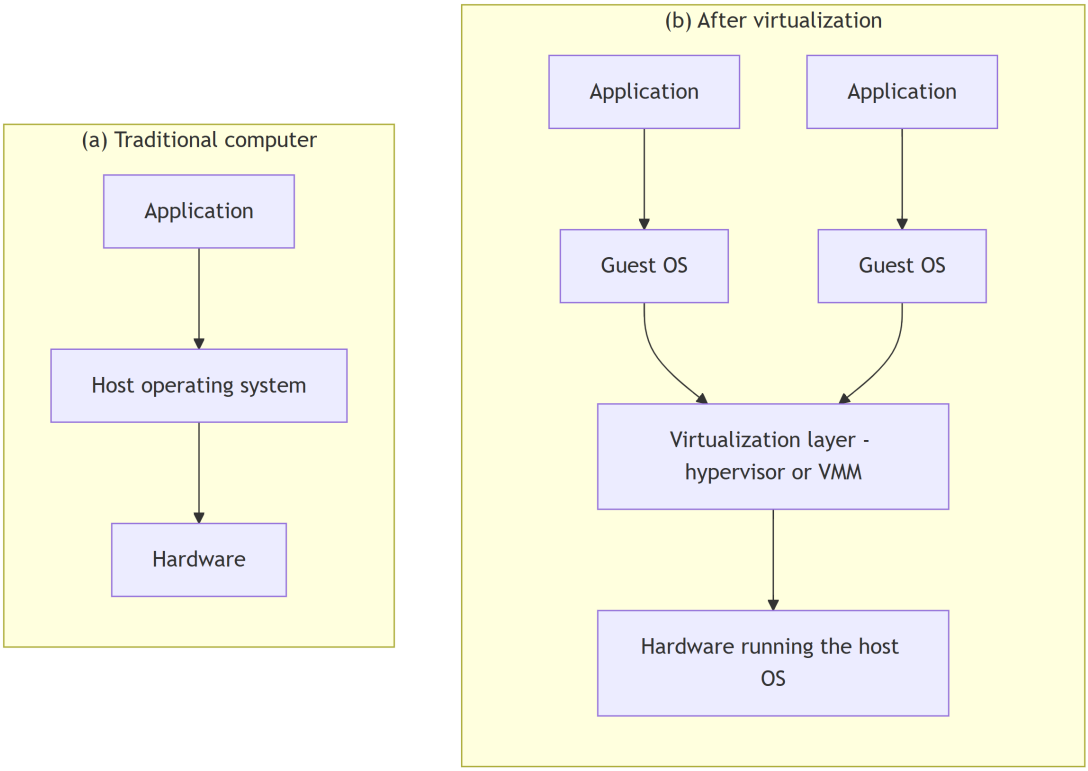
8.7 Application virtualization

The technology isolates applications from the underlying operating system and from other applications, to increase compatibility and manageability. Docker can be used for this purpose.

9. Levels of Virtualization

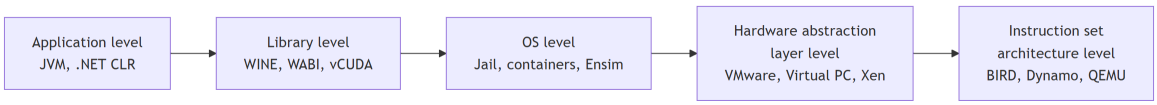
Short description: A traditional computer runs a host operating system tailored to its hardware architecture. After virtualization, different user applications managed by their own guest operating systems can run on the same hardware, independent of the host OS. This is done by adding a software layer called the virtualization layer, known as the hypervisor or virtual machine monitor.

9.1 Before and after virtualization



9.2 Implementation levels

Virtualization can be implemented at different levels within a computing system, offering varying degrees of isolation and flexibility.



9.3 Instruction set architecture level

- Virtualization is performed by emulating a given ISA with the ISA of the host machine.
- For example, MIPS binary code can run on an x86-based host machine through ISA emulation.
- It becomes possible to run a large amount of legacy binary code written for various processors on any given new hardware host.

9.4 Hardware abstraction level

- Hardware-level virtualization is performed right on top of the bare hardware. It generates a virtual hardware environment for a VM, and the process manages the underlying hardware through virtualization.
- The idea is to virtualize a computer's resources, such as its processors, memory and I/O devices, to upgrade the hardware utilization rate by multiple concurrent users.
- It was implemented in the IBM VM/370 in the 1960s. More recently the Xen hypervisor has been applied to virtualize x86-based machines running Linux or other guest OS applications.

9.5 Operating system level

- An abstraction layer between the traditional OS and user applications.
- It creates isolated containers on a single physical server, with the OS instances utilizing the hardware and software in data centres.
- The containers behave like real servers, and are commonly used for virtual hosting environments allocating hardware among mutually distrusting users.

9.6 Library support level

- Most applications use APIs exported by user-level libraries rather than lengthy system calls, so a well-documented API becomes another candidate for virtualization.
- Virtualization with library interfaces controls the communication link between applications and the rest of the system through API hooks.
- WINE implements this approach to support Windows applications on top of UNIX hosts.
- vCUDA allows applications executing within VMs to leverage GPU hardware acceleration.

9.7 User-application level

- Virtualization at the application level virtualizes an application as a VM. On a traditional OS an application runs as a process, so this is also known as process-level virtualization.
- The most popular approach is to deploy high level language (HLL) VMs. The virtualization layer sits as an application program on top of the operating system and exports the abstraction of a VM that runs programs compiled to a particular abstract machine definition.
- The Microsoft .NET CLR and the Java Virtual Machine are two good examples.

10. Containers

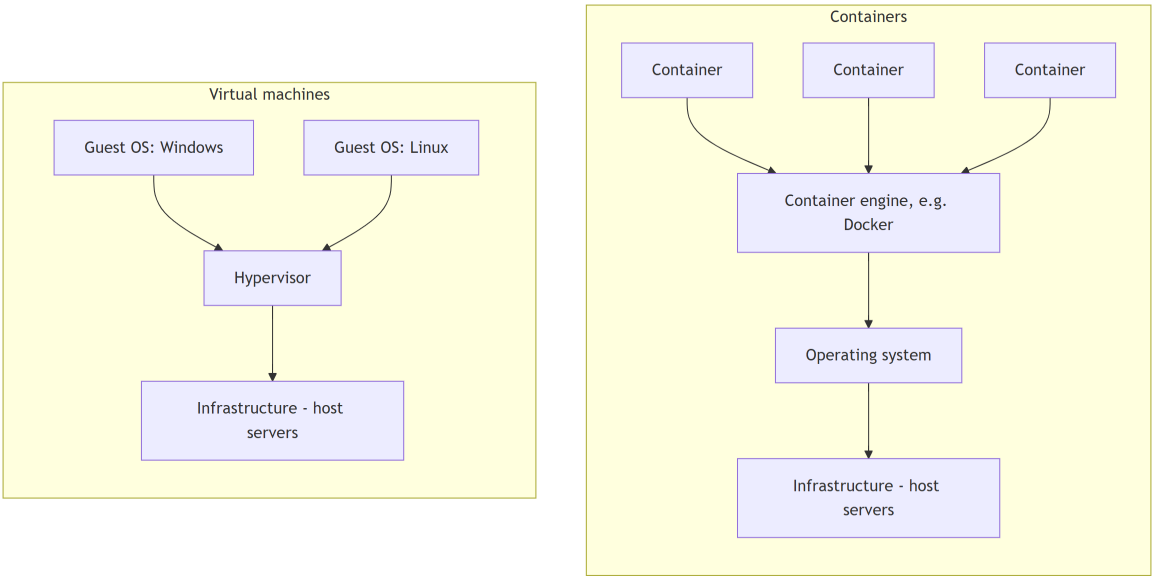
Short description: A container is an isolated, stand-alone unit that encapsulates an application and all its dependencies. It runs consistently in any environment, independent of the host system.

Theory: It is a light, executable software package that wraps everything an application needs: libraries, configuration files and binaries. The application behaves the same way on a developer's laptop, in a testing environment, or on a production server. Unlike traditional virtual machines that carry a full OS, containers pack only what is required, which makes them faster and more efficient.

How it works: Containerization is the process of packing an application together with all its dependencies into a container so it runs consistently from one computing environment to another. In simple terms it uses the host OS kernel to run many isolated instances of applications on the same machine, which makes it very lightweight and efficient.

10.1 Containers versus virtual machines

Both containers and VMs run applications in isolated environments, but they differ in architecture, performance, resource usage and startup time.



10.2 Comparison

	Containers	**Virtual machines**
Architecture	All containers share the host OS kernel; the running user spaces are isolated, which makes them lightweight.	A hypervisor running on the host OS includes a full guest OS with virtualized hardware.
Boot time	Much less, typically seconds, as no full OS has to boot.	Longer, because the full OS in the VM must be initialized.
Isolation	At the process level, less strong than a VM, though for many use cases this does not matter.	Very good, because each VM is a system on its own with its own OS.
Resource usage	Fewer resources: only the necessary binaries and libraries, not an entire OS.	Very high, as the full OS overhead is incurred for each instance.

11. Container Orchestration

Short description: Container orchestration is the process of automating the deployment, management, scaling and networking of containers throughout their lifecycle.

Theory: Orchestration platforms such as Kubernetes create a smooth, self-managing operation and coordinate microservices held in separate containers. Kubernetes eliminates many of the manual processes involved in deploying and scaling containerized applications, and Red Hat creates essential tools for securing, simplifying and automatically updating container infrastructure.

11.1 Why it is needed

When many containers run across different machines, managing them manually becomes difficult. An orchestration platform automates:

- **Deployment:** starts and distributes containers across servers.
- **Scaling:** increases or decreases the number of container instances based on demand.
- **Load balancing:** distributes incoming traffic among healthy containers.
- **Self-healing:** restarts failed containers or moves them to healthy nodes.
- **Service discovery:** allows containers to find and communicate with each other.
- **Rolling updates and rollbacks:** updates applications with minimal downtime and reverts changes if needed.
- **Resource management:** efficiently allocates CPU, memory and storage.

Benefits

Orchestration platforms automate the entire lifecycle of containers, transforming what would be a non-stop maintenance operation for IT teams into a smooth, self-managing one, and bring advantages in development methods, costs and security.